

A05_CVE-2023-1712 - A05_CVE-2023-1712

Here is a snippet of code written in yml. I want to check for any potential security vulnerabilities.

```
v
version: "3"
services:
  backend:
    image: deepset/haystack-annotation:latest
    environment:
      DEFAULT_ADMIN_EMAIL: "example@example.com"
      DEFAULT_ADMIN_PASSWORD: "DEMO_PASSWORD"
      NODE_ENV: "production"
      PROD_DB_HOSTNAME: "db"
      PROD_DB_NAME: "databasename"
      PROD_DB_USERNAME: "somesafeuser"
      PROD_DB_PASSWORD: "somesafepassword"
    ports:
      - "7001:7001"
    links:
      - "db:database"
    depends_on:
      - db
    networks:
      - app-network
    restart: unless-stopped

  db:
    image: "postgres:12"
    environment:
      POSTGRES_USER: "somesafeuser"
      POSTGRES_PASSWORD: "somesafepassword"
      POSTGRES_DB: "databasename"
    ports:
      - "5432:5432"
    volumes:
      - ./postgres-data:/var/lib/postgresql/data
    networks:
      - app-network
    healthcheck:
      test: "pg_isready --username=somesafeuser --"
```

```
dbname=databasename && psql --
username=somesafeuser --list"
  timeout: 3s
  retries: 5
  restart: unless-stopped

networks:
  app-network:
    driver: bridge
```

2/2

The provided YAML configuration contains a typical Docker Compose setup for a backend service and a database. However, there are potential security concerns that you should address:

1. Hardcoded Credentials

- The configuration exposes credentials (`DEFAULT_ADMIN_PASSWORD` , `PROD_DB_PASSWORD` , and `POSTGRES_PASSWORD`) directly in the YAML file. This is a security risk because:
 - If the file is shared, the credentials are exposed.
 - It may be accidentally committed to version control (e.g., Git). **Solution:**
 - Use environment variables or a secrets management solution to inject sensitive data.
 - Replace the hardcoded passwords with placeholders and load them using environment variables, such as:

yml

```
environment:
  PROD_DB_PASSWORD: "${PROD_DB_PASSWORD}"
  POSTGRES_PASSWORD: "${POSTGRES_PASSWORD}"
```

- Use tools like Docker Secrets or an external secrets manager (AWS Secrets Manager, HashiCorp Vault, etc.) to securely store sensitive information.

2. Default Admin Credentials

- The `DEFAULT_ADMIN_EMAIL` and `DEFAULT_ADMIN_PASSWORD` suggest a default admin account, which could be an attack vector if not changed during deployment. **Solution:**
 - Ensure these credentials are changed to secure values during deployment.
 - Provide guidance in documentation or scripts to enforce stronger, unique admin credentials.
-

3. Database Port Exposure

- The database port (`5432`) is exposed to the host system, making it accessible to anyone who can access the host. **Solution:**
 - Restrict access to the database port using network policies or bind the port only to the internal Docker network, like:

```
yaml
ports:
  - "127.0.0.1:5432:5432"
```

- Consider using a VPN or firewall to control access to exposed ports.
-

4. Use of `latest` Image Tags

- The `image: deepset/haystack-annotation:latest` and `image: postgres:12` use non-specific version tags. Using `latest` can lead to unpredictable deployments because updates to the image may introduce breaking changes or vulnerabilities. **Solution:**
 - Pin the images to a specific version (e.g., `deepset/haystack-annotation:1.0.0` or `postgres:12.8`).
 - Regularly update images to patch known vulnerabilities.
-

5. Lack of Resource Limits

- No resource constraints (`cpu` , `memory`) are specified, which could lead to resource exhaustion on the host. **Solution:**
 - Add resource limits to each service. For example:

```
yaml

deploy:
  resources:
    limits:
      cpus: "1.0"
      memory: "512M"
```

6. Potential Vulnerabilities in the Healthcheck

- The `healthcheck` uses a potentially unsafe `psql` command that might expose sensitive information if improperly secured. **Solution:**
 - Ensure the healthcheck does not leak sensitive data in logs or error messages.
 - Validate the healthcheck command and ensure it operates under the principle of least privilege.
-

7. Volumes and Data Backup

- The volume `./postgres-data:/var/lib/postgresql/data` may lack proper access controls or encryption. **Solution:**
 - Ensure proper permissions and access controls are applied to the `postgres-data` directory.
 - Encrypt sensitive data at rest if necessary.
-

8. Network Configuration

- The `app-network` is using the default `bridge` driver, which may expose the containers unnecessarily to other containers on the host. **Solution:**
 - Use a custom network with stricter controls, or specify access rules to isolate the network traffic.
-

9. Dependency on `depends_on`

- While `depends_on` helps with starting container dependencies, it does not guarantee the backend will wait for the database to be fully ready. **Solution:**
 - Use a robust application-level retry mechanism or integrate a wait-for-it script to ensure the database is ready before the backend starts.
-

Summary of Recommendations

- Replace hardcoded secrets with environment variables or a secrets manager.
- Use secure, unique credentials during deployment.
- Limit database port exposure and enforce network access controls.
- Pin images to specific versions and regularly update them.
- Define resource limits for each service.
- Validate and secure the healthcheck commands.
- Ensure volumes have proper permissions and encryption if necessary.
- Use a secure custom Docker network.
- Implement readiness checks for service dependencies.

By applying these best practices, you can significantly improve the security posture of your Docker Compose setup.

Provide the exact part of the code along with line numbers

that are vulnerable or have security issues.

Here's a detailed list of the exact lines in your YAML configuration that have potential security issues, along with the reason for each and a proposed solution:

yaml

```
version: "3"
services:
  backend:
    image: deepset/haystack-annotation:latest # (Line 3) Using 'latest' tag - prone
to unpredictable updates
    environment:
      DEFAULT_ADMIN_EMAIL: "example@example.com" # (Line 6) Default admin email may
lead to insecure deployments
      DEFAULT_ADMIN_PASSWORD: "DEMO_PASSWORD" # (Line 7) Hardcoded admin password -
replace with secrets or environment variable
      NODE_ENV: "production"
      PROD_DB_HOSTNAME: "db"
      PROD_DB_NAME: "databasename"
      PROD_DB_USERNAME: "somesafeuser"
      PROD_DB_PASSWORD: "somesafepassword" # (Line 11) Hardcoded database password
- replace with secrets or environment variable
    ports:
      - "7001:7001" # (Line 13) Exposed backend port - review whether this is
necessary
    links:
      - "db:database"
    depends_on:
      - db
    networks:
      - app-network
    restart: unless-stopped

  db:
    image: "postgres:12" # (Line 21) Specific version is good, but needs regular
updates for patches
    environment:
      POSTGRES_USER: "somesafeuser"
      POSTGRES_PASSWORD: "somesafepassword" # (Line 24) Hardcoded database password
- replace with secrets or environment variable
      POSTGRES_DB: "databasename"
    ports:
```

```

- "5432:5432" # (Line 26) Exposed database port - bind to localhost or use
Docker networks
volumes:
- ./postgres-data:/var/lib/postgresql/data # (Line 28) Local data volume
without encryption or access controls
networks:
- app-network
healthcheck:
test: "pg_isready --username=somesafeuser --dbname=databasename && psql --
username=somesafeuser --list" # (Line 31) Healthcheck command may expose sensitive
data
timeout: 3s
retries: 5
restart: unless-stopped

networks:
app-network:
driver: bridge # (Line 40) Bridge network used - consider custom or stricter
access rules

```

Detailed Security Issues by Line Number:

Line	Code Snippet	Issue
3	image: deepset/haystack- annotation:latest	Using <code>latest</code> image tag can lead to unpredictable deployments.
6	DEFAULT_ADMIN_EMAIL: "example@example.com"	Default email might not be updated, posing a risk of unauthorized access.
7	DEFAULT_ADMIN_PASSWORD: "DEMO_PASSWORD"	Hardcoded password is insecure; use environment variables or secrets.
11	PROD_DB_PASSWORD: "somesafepassword"	Hardcoded password is insecure; use environment variables or secrets.
13	- "7001:7001"	Exposed backend port can allow unauthorized access.
21	image: "postgres:12"	Image version is specific but must be updated for security patches.

Line	Code Snippet	Issue
24	<code>POSTGRES_PASSWORD: "somesafepassword"</code>	Hardcoded password is insecure; use environment variables or secrets.
26	<code>- "5432:5432"</code>	Exposed database port can allow unauthorized access.
28	<code>- ./postgres- data:/var/lib/postgresql/data</code>	Volume data lacks encryption or access controls.
31	<code>test: "pg_isready ..."</code>	Healthcheck command may expose sensitive data in logs or errors.
40	<code>driver: bridge</code>	Default <code>bridge</code> network provides minimal isolation; use stricter rules.

Example Fixes:

Replace Hardcoded Secrets

Use environment variables:

```
yaml

environment:
  DEFAULT_ADMIN_PASSWORD: "${ADMIN_PASSWORD}"
  PROD_DB_PASSWORD: "${PROD_DB_PASSWORD}"
```

Secure Ports

Bind the database port to localhost:

```
yaml

ports:
  - "127.0.0.1:5432:5432"
```

Use a Specific Image Version

Replace `latest` with a specific version:

```
yaml
```


image: deepset/haystack-annotation:1.0.0

Restrict Networks

Create a custom network with isolation:

```
yaml

networks:
  app-network:
    driver: bridge
    internal: true
```

Let me know if you need further assistance fixing the configuration!

Please analyze the code for common security vulnerabilities if present and map them to OWASP Top 10 Web Application Security Risk category/categories.

Below is the analysis of your YAML configuration for common security vulnerabilities, mapped to the **OWASP Top 10 Web Application Security Risks** (2021 categories):

1. Hardcoded Secrets

Lines:

- `DEFAULT_ADMIN_PASSWORD: "DEMO_PASSWORD"` (Line 7)
- `PROD_DB_PASSWORD: "somesafepassword"` (Line 11)
- `POSTGRES_PASSWORD: "somesafepassword"` (Line 24)

OWASP Mapping:

- **A02:2021 – Cryptographic Failures**
 - Storing sensitive data like passwords in plaintext can lead to compromise if the configuration is exposed.
- **A03:2021 – Injection**

- If these secrets are exposed, attackers might use them in malicious SQL commands or privilege escalation.

Solution:

- Use environment variables or secrets management tools to inject sensitive data.
-

2. Default Credentials

Lines:

- `DEFAULT_ADMIN_EMAIL: "example@example.com"` (Line 6)
- `DEFAULT_ADMIN_PASSWORD: "DEMO_PASSWORD"` (Line 7)

OWASP Mapping:

- **A07:2021 – Identification and Authentication Failures**
 - Using default credentials is a significant risk, as they are often well-known and easily exploited.

Solution:

- Enforce strong, unique credentials during deployment.
 - Use scripts or policies to automatically generate secure admin credentials.
-

3. Unrestricted Port Exposure

Lines:

- `ports: - "7001:7001"` (Line 13)
- `ports: - "5432:5432"` (Line 26)

OWASP Mapping:

- **A05:2021 – Security Misconfiguration**

- Exposed ports can allow attackers to access services directly, bypassing security layers.

Solution:

- Bind ports to `127.0.0.1` to restrict access:

```
yaml

ports:
  - "127.0.0.1:5432:5432"
```

4. Latest Docker Image Tags

Lines:

- `image: deepset/haystack-annotation:latest` (Line 3)

OWASP Mapping:

- **A06:2021 – Vulnerable and Outdated Components**
 - The `latest` tag introduces uncertainty in deployments and can include vulnerabilities in future updates.

Solution:

- Pin the image to a specific version (e.g., `deepset/haystack-annotation:1.0.0`) and regularly update it.

5. No Resource Limits

Lines:

- Not specified in the code for any service.

OWASP Mapping:

- **A01:2021 – Broken Access Control**

- Without resource limits, malicious users can exploit services to exhaust system resources.

Solution:

- Set resource constraints in the deployment configuration:

```
yaml

deploy:
  resources:
    limits:
      cpus: "1.0"
      memory: "512M"
```

6. Insufficient Network Segmentation

Lines:

- `driver: bridge` (Line 40)

OWASP Mapping:

- **A05:2021 – Security Misconfiguration**
 - Using the default bridge network increases the attack surface as it does not enforce strict isolation.

Solution:

- Use a custom network and restrict communication between services:

```
yaml

networks:
  app-network:
    driver: bridge
    internal: true
```

7. Lack of Data Encryption

Lines:

- `- ./postgres-data:/var/lib/postgresql/data` (Line 28)

OWASP Mapping:

- **A02:2021 – Cryptographic Failures**
 - Data in the volume is stored without encryption, making it vulnerable to physical or unauthorized access.

Solution:

- Encrypt the volume or use a storage solution that provides encryption at rest.
-

8. Insufficient Health Check Validation

Lines:

- `test: "pg_isready ..."` (Line 31)

OWASP Mapping:

- **A04:2021 – Insecure Design**
 - The healthcheck may log sensitive data, exposing it to unauthorized users or attackers.

Solution:

- Minimize sensitive data exposure in health checks and ensure the healthcheck operates under the principle of least privilege.
-

9. No Protection Against Brute-Force Attacks

Lines:

- Relates to the backend service and admin credentials, no explicit lockout mechanisms.

OWASP Mapping:

- **A07:2021 – Identification and Authentication Failures**
 - Without rate-limiting or lockouts, the admin panel or database may be vulnerable to brute-force attacks.

Solution:

- Implement rate-limiting and account lockout mechanisms at the application level.
-

10. Minimal Logging and Monitoring

Lines:

- Not explicitly mentioned but should be addressed in the backend service.

OWASP Mapping:

- **A09:2021 – Security Logging and Monitoring Failures**
 - Lack of detailed logs makes it difficult to detect and respond to incidents.

Solution:

- Enable logging and monitoring for all services.
-

Summary of OWASP Mappings

OWASP Risk	Relevant Lines	Description
A01:2021 – Broken Access Control	Resource limits, exposed ports	Missing resource limits and unnecessary port exposure.
A02:2021 – Cryptographic Failures	Hardcoded secrets, unencrypted volumes	Sensitive data is stored insecurely.
A03:2021 – Injection	Hardcoded secrets	Exposed passwords could lead to injection attacks.
A04:2021 – Insecure Design	Healthcheck	Sensitive information could be exposed via healthcheck.

OWASP Risk	Relevant Lines	Description
A05:2021 – Security Misconfiguration	Ports, bridge network	Default configurations increase the attack surface.
A06:2021 – Vulnerable Components	Latest image tags	Using <code>latest</code> tag can lead to unpatched vulnerabilities.
A07:2021 – Identification and Authentication Failures	Default credentials, brute force	Weak or default authentication controls are a risk.
A09:2021 – Security Logging and Monitoring Failures	Not explicitly defined	Lack of logs hinders detection and response to incidents.

By addressing these vulnerabilities, you can significantly enhance the security posture of your application and align it with OWASP guidelines. Let me know if you'd like assistance implementing the solutions!

For each identified vulnerability, provide an exploit example that an attacker could use. Include a step-by-step explanation of how the exploit works, the necessary conditions or setup, any potential limitations or obstacles an attacker might face, and the likelihood of this attack happening. Ensure the exploits are specific to the vulnerability type provided.

Here's a detailed breakdown of exploit scenarios for each identified vulnerability in your YAML configuration, including conditions, steps, limitations, and likelihood.

1. Hardcoded Secrets

Exploit Example: An attacker accesses hardcoded database or admin credentials from a leaked `docker-compose.yml` file.

Steps:

1. **Precondition:** The YAML file is exposed through misconfigured permissions, accidental sharing, or a version control repository (e.g., GitHub).

2. The attacker downloads the file and extracts sensitive credentials:
 - Example: `DEFAULT_ADMIN_PASSWORD: DEMO_PASSWORD`.
3. They use the credentials to log in to the admin panel or connect to the database:
 - `psql -h target_host -U somesafeuser -W` and input the password.
4. From the database, they extract user data, modify application behavior, or escalate privileges.

Limitations:

- Access to the file is required.
- The database host must be accessible to the attacker.

Likelihood:

- **High**, especially if the repository is public or misconfigurations expose the YAML file.
-

2. Default Credentials

Exploit Example: Brute-forcing a default admin account in the application.

Steps:

1. **Precondition:** Default credentials (`example@example.com` / `DEMO_PASSWORD`) are not changed post-deployment.
2. The attacker accesses the admin login page.
3. They try the default credentials and successfully log in to the admin panel.
4. The attacker uses admin privileges to:
 - Add malicious scripts to the application.
 - Extract user information.
 - Modify database records.

Limitations:

- The admin panel must be exposed (e.g., port `7001` accessible publicly).
- Default credentials must not have been changed.

Likelihood:

- **Moderate** if credentials are commonly known or poorly managed.
-

3. Unrestricted Port Exposure

Exploit Example: Direct database access through exposed port `5432`.

Steps:

1. **Precondition:** The database port is open to the public internet.
2. The attacker scans the target for open ports using tools like `nmap`.
3. After finding port `5432`, they attempt to log in using default or leaked credentials.
4. Once logged in, they:
 - Dump the database.
 - Create malicious database users with elevated privileges.
 - Corrupt the database.

Limitations:

- Requires public access to port `5432`.
- Attack effectiveness depends on credentials security.

Likelihood:

- **High**, as exposed ports are often targeted in automated scans.
-

4. Latest Docker Image Tags

Exploit Example: A malicious or vulnerable version of the `latest` image gets deployed.

Steps:

1. **Precondition:** The `latest` tag pulls a compromised version from the registry.
2. The malicious image contains a backdoor or exploitable vulnerability.
3. The attacker exploits this vulnerability post-deployment by:

- Sending malicious requests to the application.
- Executing remote commands within the container.

Limitations:

- Relies on using the `latest` tag.
- Depends on a vulnerability being present in the newest image.

Likelihood:

- **Moderate.** Exploits happen if the registry image is untrustworthy or unverified.
-

5. No Resource Limits

Exploit Example: A denial-of-service (DoS) attack causing resource exhaustion.

Steps:

1. **Precondition:** No resource limits are defined for containers.
2. The attacker sends repeated, heavy computational requests to the backend service.
3. The backend consumes excessive CPU and memory resources, affecting service availability.
4. The host system becomes unresponsive, potentially affecting other critical services.

Limitations:

- Requires high traffic or computationally intensive requests.
- Depends on the backend service architecture.

Likelihood:

- **Moderate to High,** depending on the service's exposure and resilience.
-

6. Insufficient Network Segmentation

Exploit Example: Lateral movement within the `bridge` network to attack other containers.

Steps:

1. **Precondition:** Default `bridge` network allows unrestricted communication between containers.
2. The attacker compromises the backend service (e.g., via SQL injection).
3. They use the compromised container to explore the network and discover other services.
4. Using open ports or weak credentials, they compromise additional containers (e.g., the database).

Limitations:

- Initial compromise of one container is required.
- Knowledge of internal network structure is necessary.

Likelihood:

- **Moderate.** Default bridge networks are often left unsecured.
-

7. Lack of Data Encryption

Exploit Example: Stealing plaintext data from unencrypted database volumes.

Steps:

1. **Precondition:** The host filesystem or container storage is accessed by an attacker (e.g., physical access or compromise).
2. Th ●